

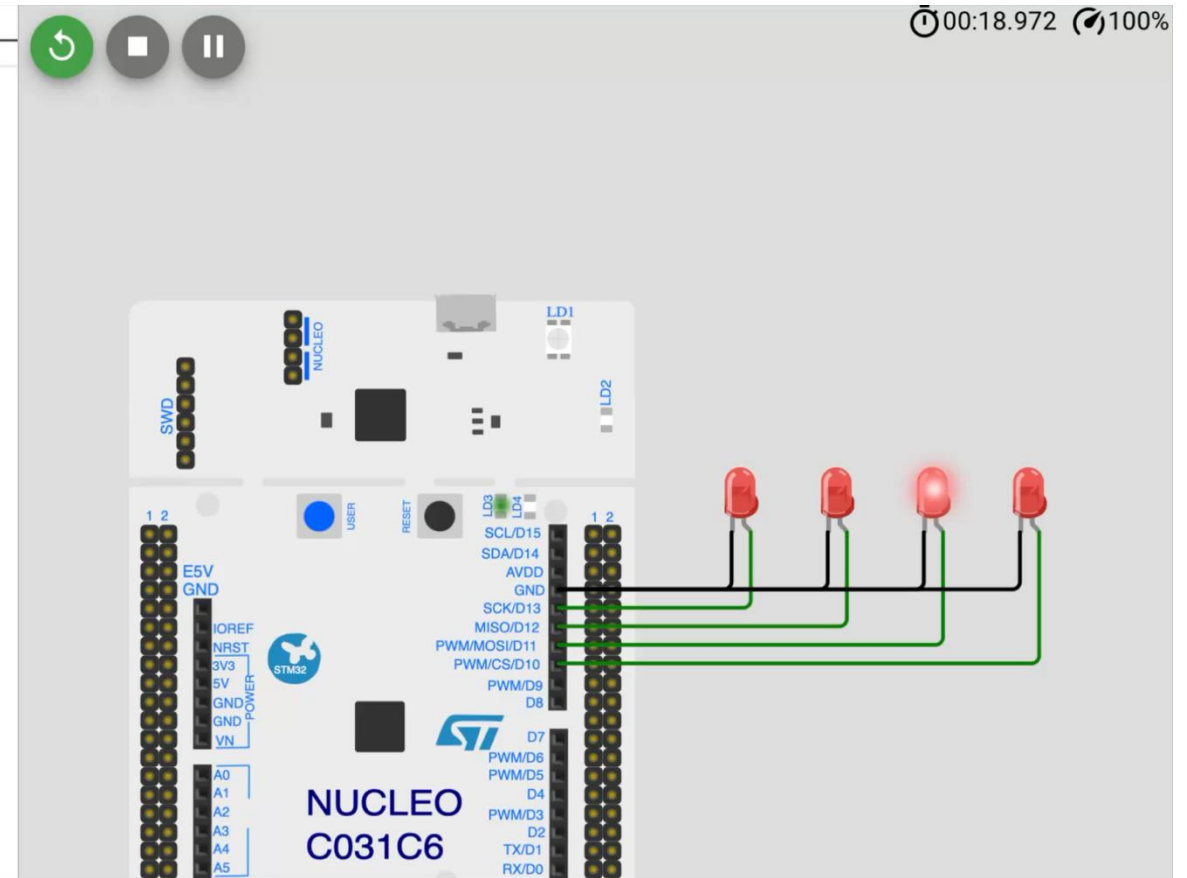
I/O ports, switch and LED interfacing

Yuning You

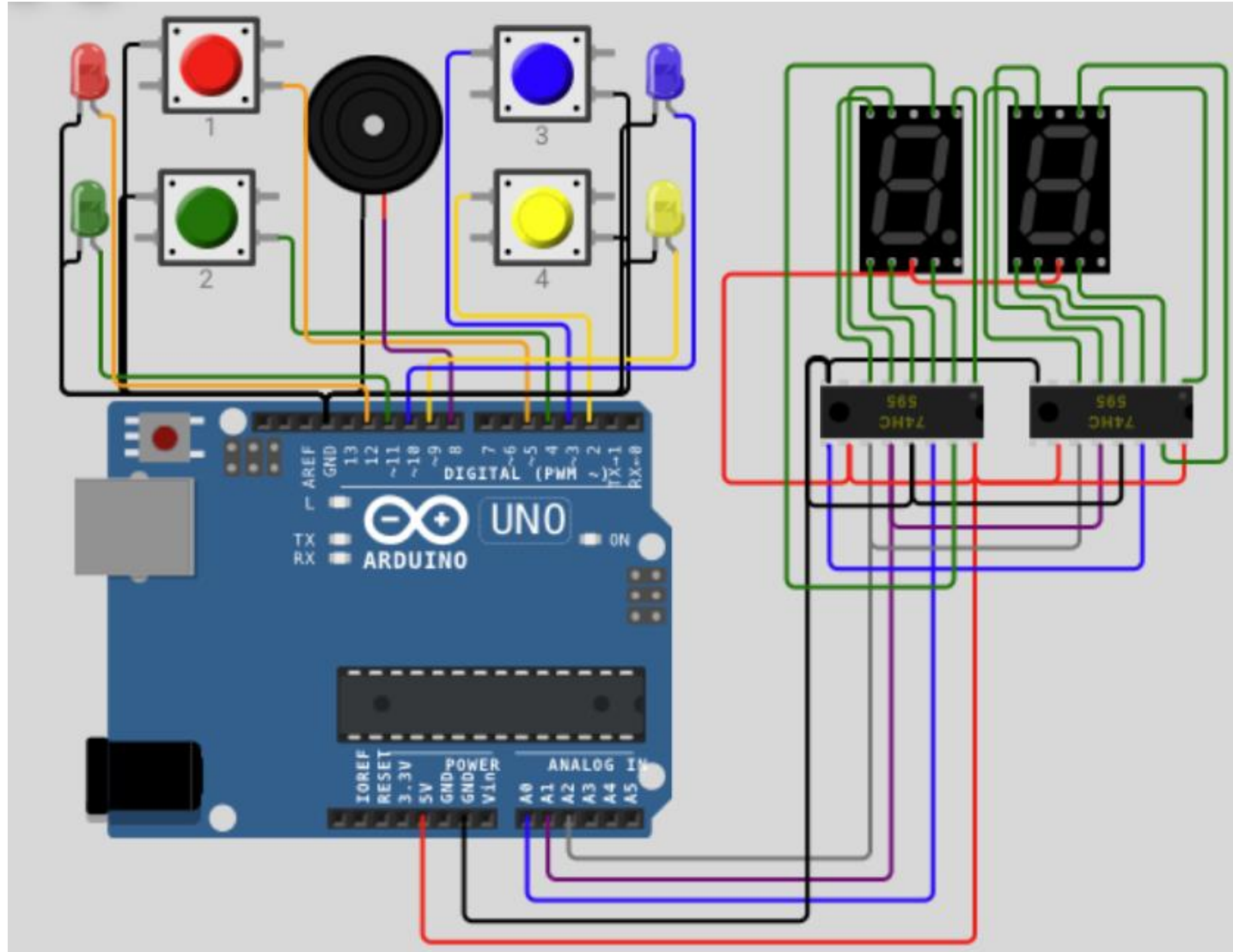
School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

Recap: Running LEDs

```
1 // 4-bit binary counter using LEDs (each LED toggles at different rates)
2
3 int ledPin[] = {13, 12, 11, 10};           // LED pins: MSB → LSB
4 int ledDelay[] = {8000, 4000, 2000, 1000}; // toggle interval (ms)
5 unsigned long lastChangeTime[4];          // store last toggle time of each LED
6 int ledState[4] = {LOW, LOW, LOW, LOW};   // store current state (LOW/HIGH)
7 unsigned long currTime;                   // current time variable
8
9 void setup() {
10     currTime = 0;                         // initialize timer
11     for (int i = 0; i < 4; i++) {
12         pinMode(ledPin[i], OUTPUT);        // set pin as output
13         digitalWrite(ledPin[i], ledState[i]); // turn LED off initially
14         lastChangeTime[i] = millis();      // record initial time
15     }
16 }
17
18 void loop() {
19     currTime = millis();                  // get current time
20
21     for (int i = 0; i < 4; i++) {
22         // check all LEDs
23         if (currTime - lastChangeTime[i] >= ledDelay[i]) {
24             // time to toggle the LED
25             ledState[i] = !ledState[i];    // flip LED state
26             digitalWrite(ledPin[i], ledState[i]); // apply new state
27             lastChangeTime[i] = currTime;  // reset timer for that LED
28         }
29     }
30 }
```

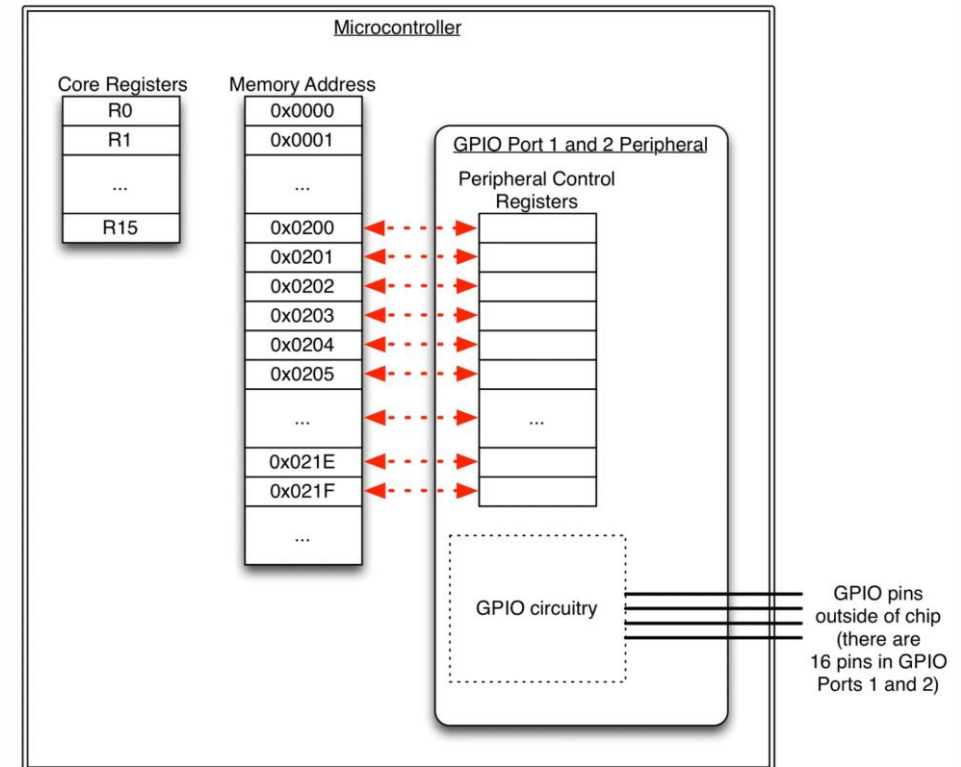


Recap: Simon game



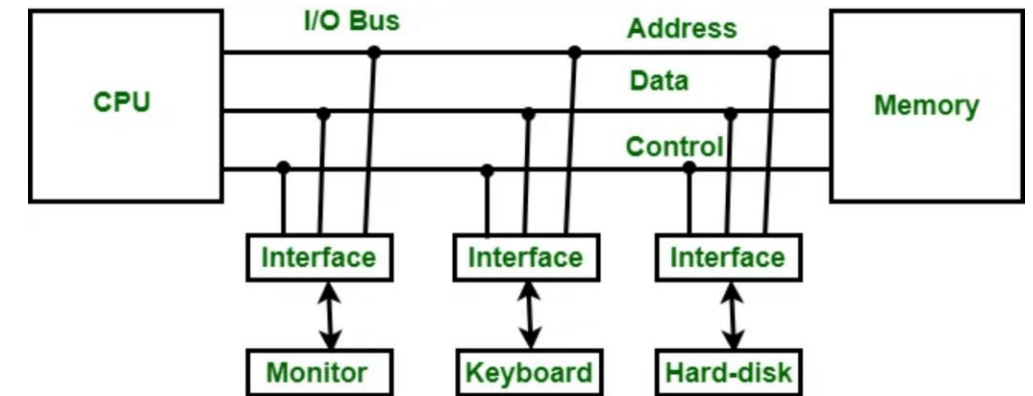
Learning Objectives for This Lecture

- 1 Understand parallel port architecture and memory-mapped I/O access mechanisms
- 2 Master GPIO configuration including direction, digital enable, and alternate function registers
- 3 Design and implement practical switch and LED interfaces with proper debouncing and current limiting



What is a Parallel I/O Port?

- 1 A parallel I/O port is a set of digital pins that can be independently configured as inputs or outputs.
- 2 Ports are organized in groups (Port A, Port B, etc.), with each port containing multiple pins.
- 3 Memory-mapped I/O allows software to control ports by reading and writing to specific memory addresses.



I/O Bus Architecture

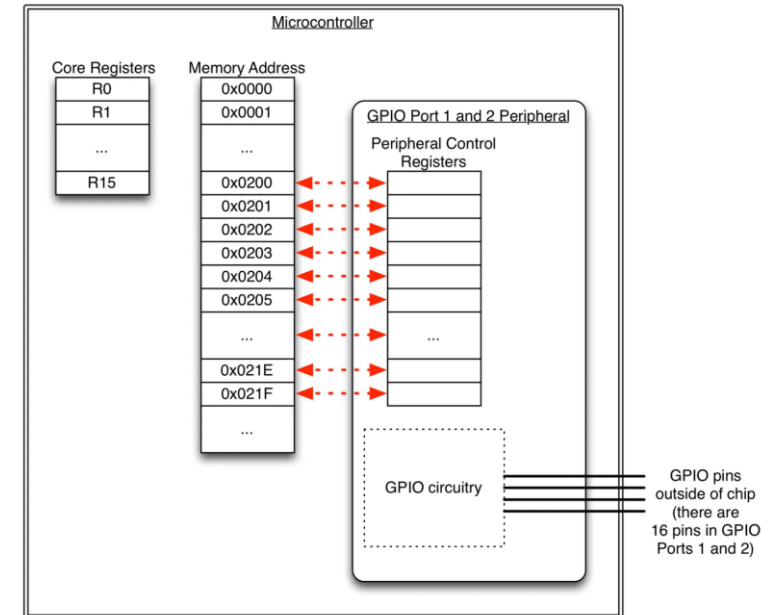
GPIO and Alternative Functions

- 1 Each pin can function as GPIO (General Purpose I/O) or be assigned to alternative functions like UART, SPI, I2C, or timers.
- 2 Register specifies which of up to 8 alternative functions a pin performs.
- 3 Proper configuration ensures pins are used for their intended purpose without conflicts.

Architectural Insight

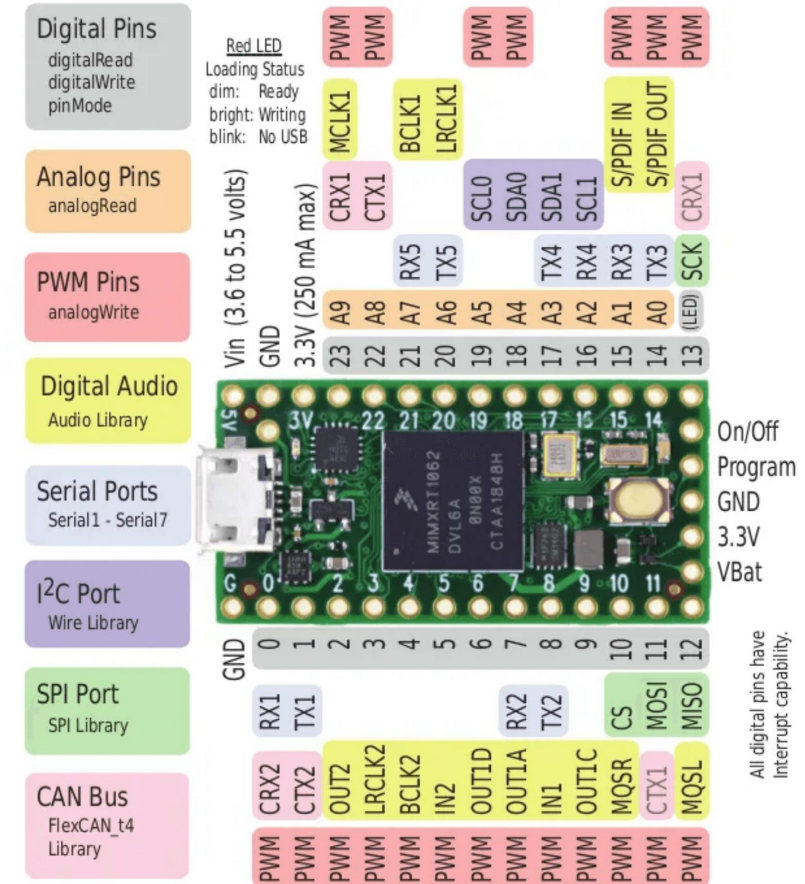
The same physical pin connects to multiple internal modules. The multiplexer logic, controlled by PCTL, determines which module drives the pin.

Internal Architecture



TM4C123 Port Organization

- 1 The TM4C123 has 43 I/O lines organized into 6 ports (A through F).
- 2 Each port has 8 pins (PA0-PA7, PB0-PB7, etc.), though not all pins are always available.
- 3 Ports are connected to both the Advanced Peripheral Bus (APB) and Advanced High-Performance Bus (AHB) for flexibility.



- 1 Each port has associated registers at specific memory addresses for data, direction, and control.
- 2 Reading and writing to these addresses directly controls the corresponding I/O pins.
- 3 This approach unifies I/O operations with memory operations, simplifying the instruction set.



Essential I/O Registers for GPIO

GPIO_PORT_DATA_R

Data Register

Contains the current logic state of all pins in the port.

Read to get input values; Write to set output values.

GPIO_PORT_DIR_R

Direction Register

Specifies whether each pin is an Input or Output.

0 = Input (Read), 1 = Output (Write).

GPIO_PORT_DEN_R

Digital Enable

Activates digital functionality for each pin.

Must be set to 1 for digital I/O; 0 disables the digital buffer.

Steps to Initialize a GPIO Port

Proper initialization is critical. Failing to follow this sequence often results in the "Bus Fault" error or unresponsive pins. This sequence is often referred to as the "Initialization Ritual".

01

Clock Activation

Enable the clock for the specific port in the **RCGCGPIO** register. Wait for the clock to stabilize.

02

Function Selection

Disable analog mode (**AMSEL**) and select the GPIO function (**PCTL & AFSEL**).

03

Pin Setup

Set the direction (**DIR**) and enable the digital I/O buffer (**DEN**).

Configuring Pin Functions with AFSEL and PCTL

GPIO_PORT_AFSEL_R

The Gatekeeper

Determines whether the pin functions as a standard GPIO or uses an alternative hardware peripheral.

0 = GPIO Mode (Default)

1 = Alternate Function Mode

GPIO_PORT_PCTL_R

The Selector

Selects **which** specific peripheral signal is connected to the pin when AFSEL is set to 1. Uses a 4-bit encoding (PMC) for each pin.

0x0 = GPIO

0x1 = UART

0x2 = SSI

DEN and AMSEL Registers

GPIO_PORT_DEN_R

Digital Enable

Controls the digital I/O buffer. Bit = 1: digital enabled. Bit = 0: digital disabled (tristate).

GPIO_PORT_AMSEL_R

Analog Mode Select

Controls the analog isolation circuit. Bit = 1: analog enabled. Bit = 0: analog disabled.

Configuration Logic Matrix

Desired Mode	DEN Bit	AMSEL Bit	Result
Digital I/O	1	0	Standard GPIO operation (Input or Output)
Analog Function	0	1	ADC or Analog Comparator input (Low power leakage)

Bit-Specific Addressing

- 1 Allows software to access individual bits of a port without affecting others, eliminating the need for read-modify-write sequences.
- 2 The address lines [9:2] act as a mask; only bits corresponding to set address lines are modified during a write.
- 3 This hardware feature ensures atomic operations, making GPIO manipulation interrupt-safe and more efficient.

Address Calculation Logic

Target: Write to Pin 5 (Bit 5)
Mask: $(1 \ll 5) = 0x20$
Offset: $\text{Mask} \ll 2 = 0x80$
Address: $\text{Base} + 0x80$

Implementation Comparison

```
// Standard (RMW - Not Atomic)
GPIO_PORTA_DATA_R |= 0x20;

// Bit-Specific (Atomic)
(*(volatile uint32_t *)0x40004080) = 0x20;
```

Reading Digital Input from GPIO Pins

Set Direction

01

Configure the pin as an input by clearing the corresponding bit in the GPIO Direction (DIR) register.

```
GPIO_PORTF_DIR_R &= ~0x10; // PF4 Input
```

Enable Digital

02

Enable the digital function for the pin by setting the bit in the GPIO Digital Enable (DEN) register.

```
GPIO_PORTF_DEN_R |= 0x10; // Enable PF4
```

Read Data

03

Read the GPIO Data register. Use bitwise masking to isolate the specific pin's value from the port data.

```
in = GPIO_PORTF_DATA_R & 0x10;
```

Writing Digital Output to GPIO Pins

01

Set Direction

Configure the pin as an output by **setting** the corresponding bit in the GPIO Direction (DIR) register to 1.

```
GPIO_PORTF_DIR_R |= 0x02;  
// PF1 Output (Red LED)
```

02

Enable Digital

Enable the digital function for the pin by setting the bit in the GPIO Digital Enable (DEN) register.

```
GPIO_PORTF_DEN_R |= 0x02;  
// Enable PF1 Digital
```

03

Write Data

Write to the GPIO Data register. Writing a 1 drives the pin High (3.3V); writing a 0 drives it Low (0V).

```
GPIO_PORTF_DATA_R |= 0x02; // High  
GPIO_PORTF_DATA_R &= ~0x02; // Low
```

Complete GPIO Initialization Example

1 Clock Activation

Enable the clock for Port F (Bit 5) and wait for stabilization.

2 Direction Setup

Set PF1, PF2, and PF3 as outputs (Bit=1) for LEDs.

3 Digital Enable

Enable digital functionality for the selected pins.

```
void PortF_Init(void) {  
    volatile uint32_t delay;  
  
    // 1. Activate Clock for Port  
    SYSCTL_RCGCGPIO_R |= 0x20;  
  
    // 2. Allow time for clock to stabilize  
    delay = SYSCTL_RCGCGPIO_R;  
  
    // 3. Set Direction: PF3-1 as Output  
    (0x0E)GPIO_PORTF_DIR_R |= 0x0E;  
  
    // 4. Enable Digital: PF3-1  
    1GPIO_PORTF_DEN_R |= 0x0E;  
}
```


I/O Port Alternative Functions

To maximize functionality within a limited pin count, most microcontroller pins are **multiplexed**. They can serve as standard GPIO or perform specialized hardware tasks.



Universal Asynchronous Receiver/Transmitter

Serial communication (PC, GPS, Bluetooth).



Synchronous Serial Interface (SPI)

High-speed data transfer for displays and storage.



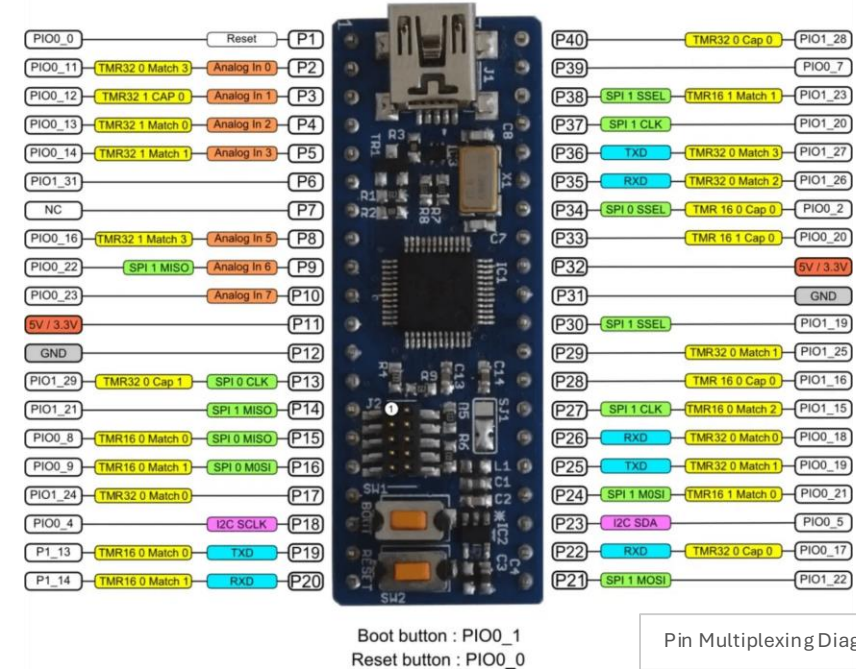
Inter-Integrated Circuit

Two-wire interface for sensors and peripherals.



Pulse Width Modulation

Motor control, LED dimming, audio.



UART Configuration on I/O Pins

Target: Configure **UART0** on Port A

PA0 (Rx) & PA1 (Tx)

1. Enable Alt Function

Set the AFSEL bits for PA0 and PA1 to disconnect them from the standard GPIO data register and route them to the alternate hardware.

```
GPIO_PORTA_AFSEL_R |= 0x03;
```

2. Select Peripheral

Configure the Port Control (PCTL) register. For Port A, the code 0x1 maps to UART0.

PMC1 (PA1): 1

PMC0 (PA0): 1

1

1

```
GPIO_PORTA_PCTL_R = (GPIO_PORTA_PCTL_R &  
0xFFFFF00) | 0x11;
```

3. Enable Digital

Even for alternate functions like UART, the digital I/O buffer must be enabled for the signal to pass through.

```
GPIO_PORTA_DEN_R |= 0x03;
```

Introduction to Switch Interfacing

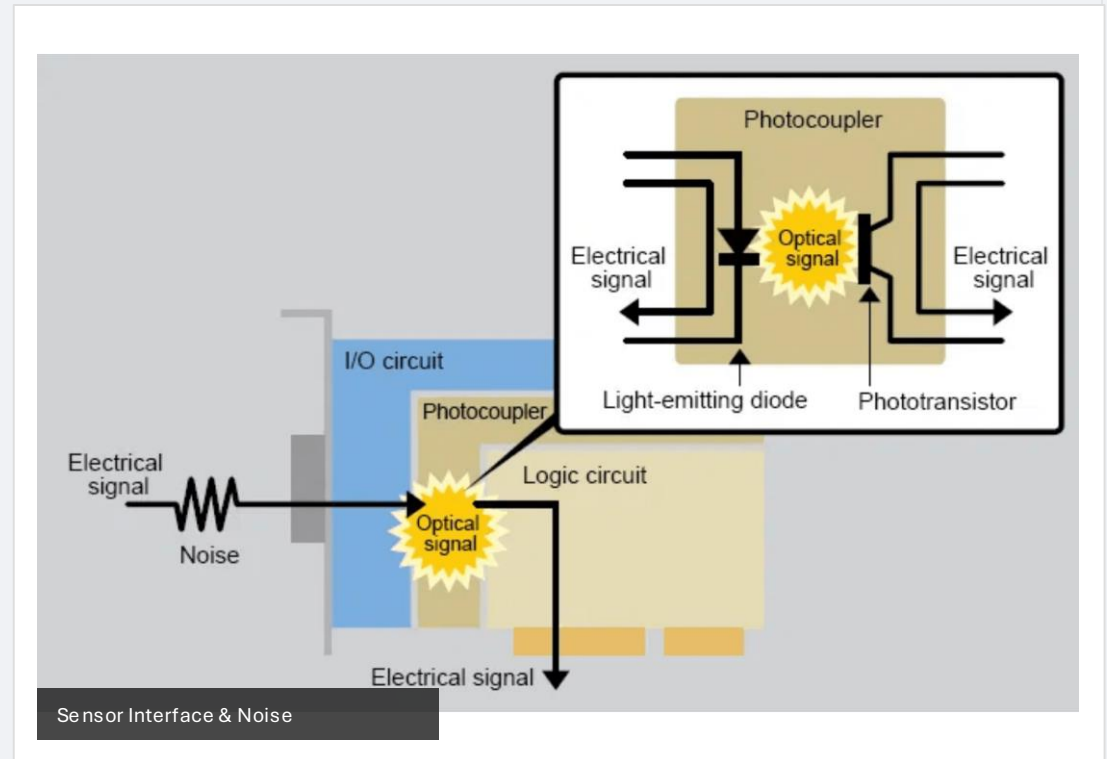
A switch converts a physical action into an electrical signal. It is a binary sensor: **Open** (no connection) or **Closed** (connection).

! The Floating State

When open, the input can float and give unpredictable logic levels.

! Signal Noise

Contacts can bounce, causing multiple transitions on a single press.



Pull-Down and Pull-Up Resistor Circuits

Positive Logic (Pull-Down) ↓

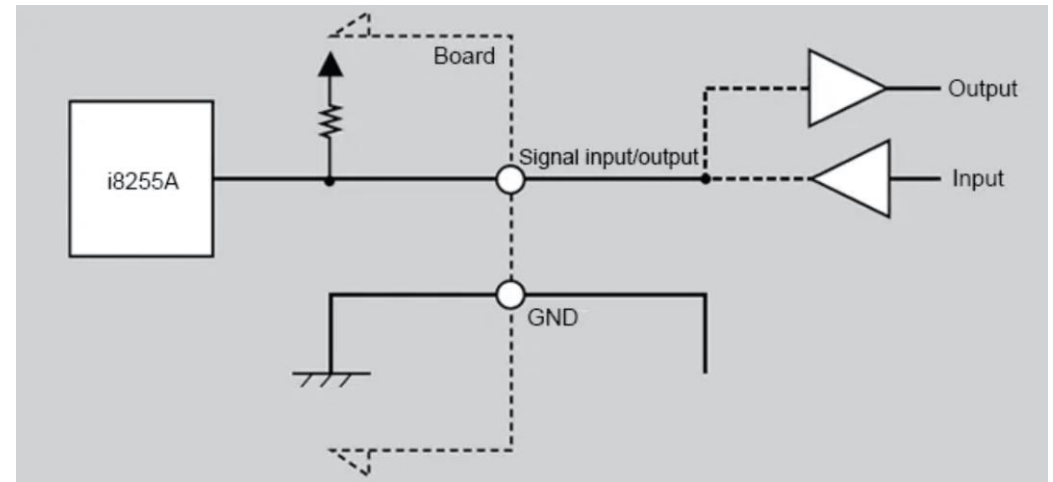
The resistor connects the pin to Ground (GND). The switch connects the pin to Vcc (3.3V).

Switch Open (Released)	0 (Low)
Switch Closed (Pressed)	1 (High)

Negative Logic (Pull-Up) ↑

The resistor connects the pin to Vcc (3.3V). The switch connects the pin to Ground (GND).

Switch Open (Released)	1 (High)
Switch Closed (Pressed)	0 (Low)



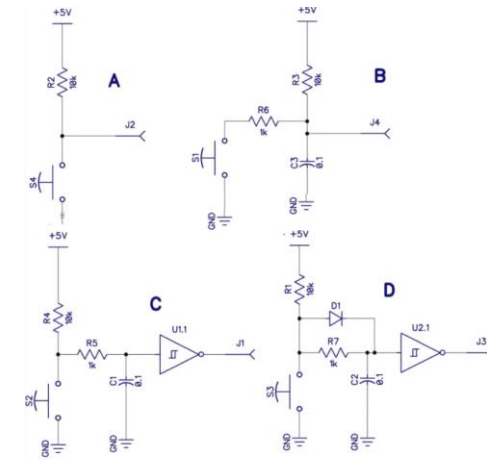
Circuit diagrams illustrating the connection of mechanical switches using pull-up and pull-down resistors to define logic levels.

Contact Bounce and Why Debouncing is Necessary

The Mechanical Reality

Switch contacts have mass and elasticity. When pressed they often make several brief connections before settling, creating rapid transitions.

- **Duration:** Typically 10–20 ms.
- **The Noise:** Rapid, unpredictable High/Low transitions.
- **System Speed:**
An MCU can execute many instructions during bounce.



Digital Consequence

User Action: 1 Press

CPU Sees: 10+ Interrupts

Result: System Malfunction

Implementing Debouncing in Software

1. The "Wait-and-See" Method

Simple but blocking. When a state change is detected, the software waits for the bounce duration (10-20ms) to pass before reading the final state.

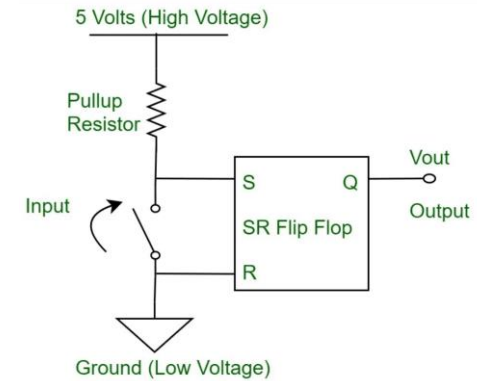
```
if (Input != LastState) {  
    Delay(10ms);  
    CurrentState = ReadInput();  
}
```

2. The Counter/Voting Method

Robust and non-blocking. The input is sampled periodically (e.g., every 1ms). The state is only updated if N consecutive samples are consistent.

```
if (Input == High) Count++; else Count = 0;  
if (Count >= Threshold) State = High;
```

Hardware Alternative



SR Latch Circuit: Uses a cross-coupled NAND gate latch to physically filter noise. While it eliminates software overhead, it increases component count and board size.

Building a Switch Interface Circuit

Bill of Materials

Tactile Switch (4-pin)

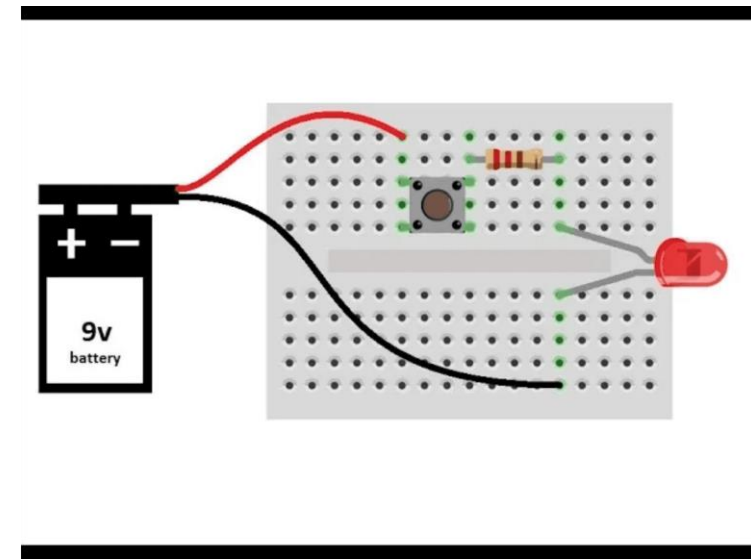
10k Ω Resistor

Jumper Wires

Breadboard

Wiring Steps (Positive Logic)

- 1 Insert the switch across the central ravine of the breadboard.
- 2 Connect one leg of the switch directly to the **3.3V (Vcc)** rail.
- 3 Connect the other leg to the **GPIO Pin** (e.g., PF4).
- 4 Connect the **10k Ω Resistor** from the GPIO leg to **Ground (GND)**. This acts as the pull-down.



Lab Tip

Verify your resistor value! A 10k Ω resistor typically has the color bands **Brown-Black-Orange**. Using a value too low (e.g., 10 Ω) causes excessive current; too high makes the signal weak.

Initializing a Switch Input

1. Direction (DIR)

Clear the bit to 0 to configure the pin as an **Input**.

2. Pull-Up Resistor (PUR)

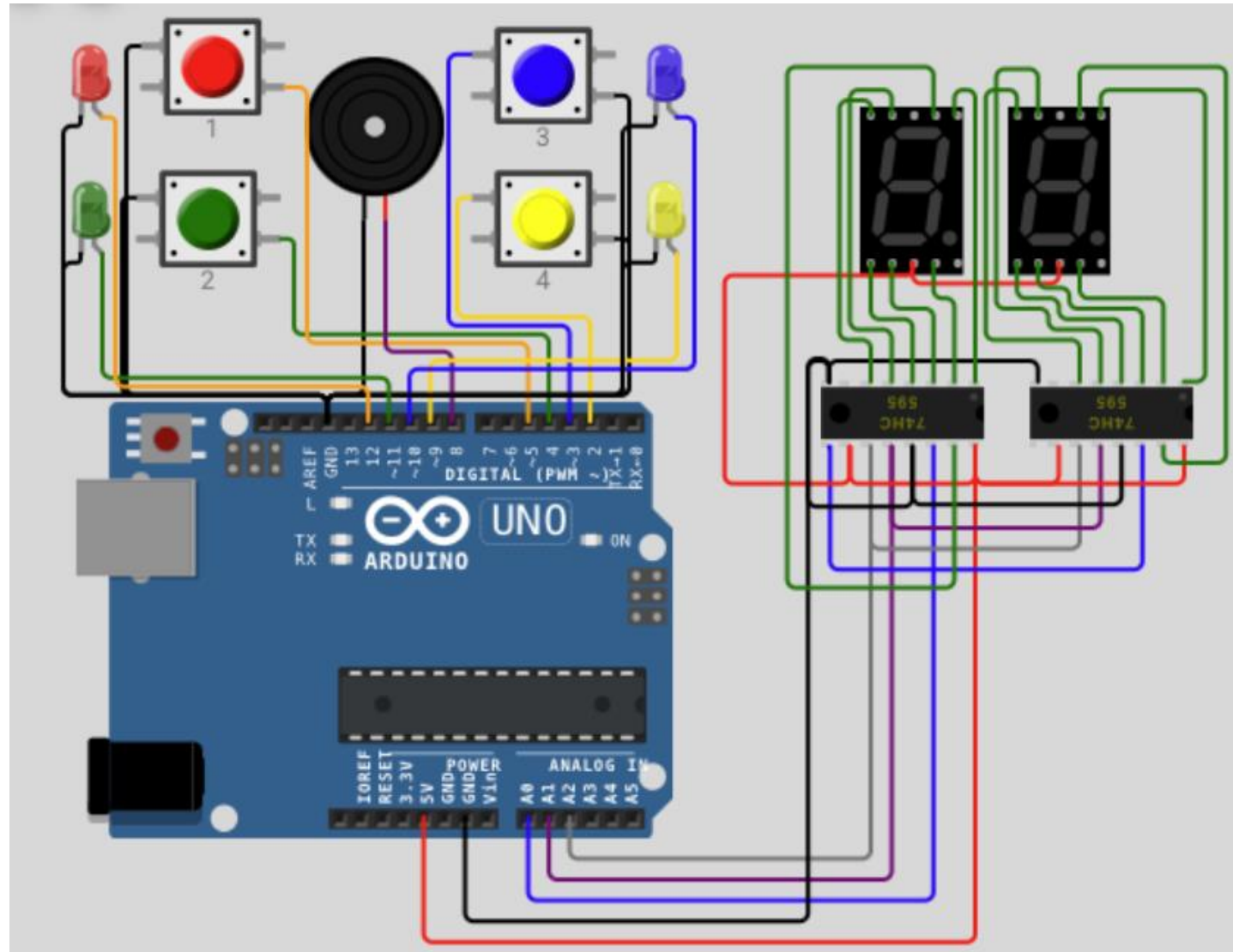
Set the bit to 1 to enable the internal weak pull-up resistor. Essential for negative logic switches.

3. Digital Enable (DEN)

Set the bit to 1 to enable the digital input buffer.

```
void Switch_Init(void) {  
    volatile uint32_t delay;  
  
    // 1. Activate Clock for Port  
    FSYSCTL_RCGCGPIO_R |= 0x20;  
    delay = SYSCTL_RCGCGPIO_R;  
  
    // 2. Set Direction: PF4 as Input  
    (0)GPIO_PORTF_DIR_R &= ~0x10;  
  
    // 3. Enable Pull-Up Resistor for  
    PF4GPIO_PORTF_PUR_R |= 0x10;  
  
    // 4. Enable Digital:  
    PF4GPIO_PORTF_DEN_R |= 0x10;  
}
```


Virtual lab session recap: Simon game



Virtual lab session: Simon game

- Can you implement a debouncing logic during button pressing (switch)?

1. The "Wait-and-See" Method

Simple but blocking. When a state change is detected, the software waits for the bounce duration (10-20ms) to pass before reading the final state.

```
if (Input != LastState) {  
    Delay(10ms);  
    CurrentState = ReadInput();  
}
```

Virtual lab session: Simon game

- Can you implement a debouncing logic during button pressing (switch)?

2. The Counter/Voting Method

Robust and non-blocking. The input is sampled periodically (e.g., every 1ms).

The state is only updated if N consecutive samples are consistent.

```
if (Input == High) Count++; else Count = 0;  
if (Count >= Threshold) State = High;
```

Virtual lab session: Simon game

- Can you mimic a broken button (switch)—one that sometimes doesn't respond when you press it?

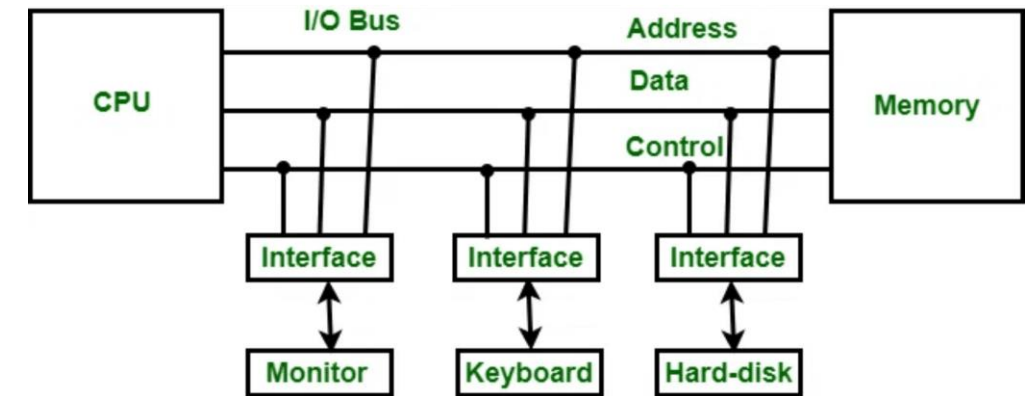
I/O ports, switch and LED interfacing

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

Recap: What is a Parallel I/O Port?

- 1 A parallel I/O port is a set of digital pins that can be independently configured as inputs or outputs.
- 2 Ports are organized in groups (Port A, Port B, etc.), with each port containing multiple pins.
- 3 Memory-mapped I/O allows software to control ports by reading and writing to specific memory addresses.



Recap: Introduction to Switch Interfacing

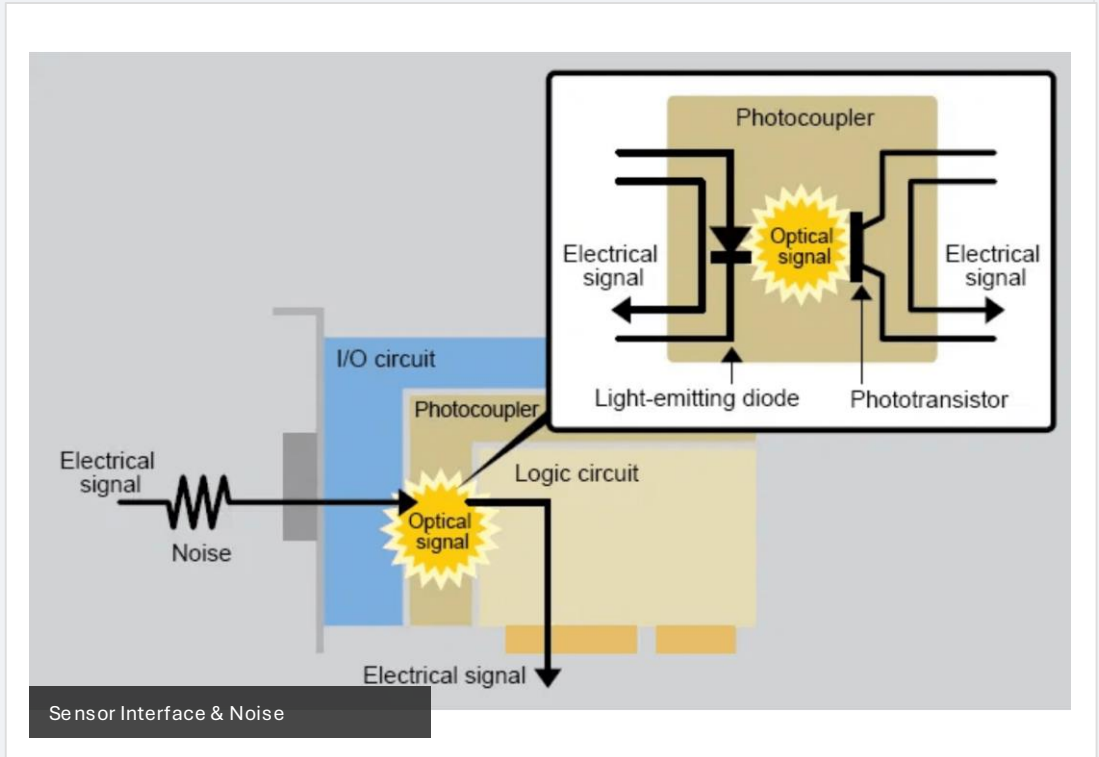
A switch converts a physical action into an electrical signal. It is a binary sensor: **Open** (no connection) or **Closed** (connection).

! The Floating State

When open, the input can float and give unpredictable logic levels.

! Signal Noise

Contacts can bounce, causing multiple transitions on a single press.



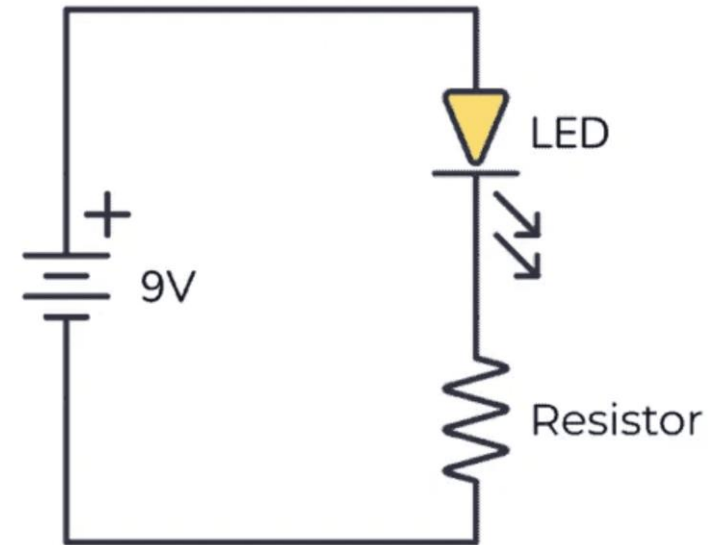
Introduction to LED Interfacing

The Diode Nature

An LED is a polarized component: current flows from the **Anode (+)** to the **Cathode (-)**. If reversed, it will not light.

⚠ Current Limiting

LEDs have a non-linear IV curve: a small voltage increase past the forward voltage causes a large current rise. **Rule:** Never connect an LED directly to a source; always use a series resistor to limit current.



Basic LED Circuit with Series Resistor

Calculating LED Current-Limiting Resistors

LEDs are non-linear devices. Connecting them directly to a voltage source causes excessive current flow and failure. A series resistor is required to limit this current to a safe level.

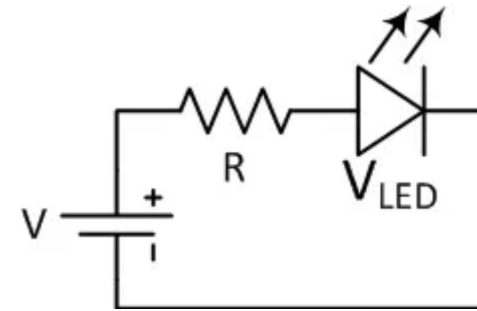
Ohm's Law for LED Circuits

$$R = (V_{CC} - V_{LED}) / I_{LED}$$

V_{CC} : Supply Voltage (e.g., 3.3V)

V_{LED} : Forward Voltage Drop (~1.6V Red)

I_{LED} : Desired Current (e.g., 10mA)



Design Example

Source: 3.3V

LED: Red (1.6V drop)

Target: 10mA (0.01A)

$(3.3 - 1.6) / 0.01$

= 170 Ω

Select nearest standard value: **180 Ω**

LED Configuration Modes

↑ Active High

The microcontroller pin drives the Anode. The Cathode connects to Ground. The pin must supply (source) current to light the LED.

Path: Pin → Resistor → LED → GND

Logic Level	Voltage	LED State
1	3.3V	ON
0	0V	OFF

↓ Active Low

The microcontroller pin connects to the Cathode. The Anode connects to Vcc. The pin must absorb (sink) current to light the LED.

Path: Vcc → Resistor → LED → Pin

Logic Level	Voltage	LED State
1	3.3V	OFF
0	0V	ON

Implementing LED Output Control

1. Direction (DIR)

Set the bit to 1 to configure the pin as an **Output**. This enables the output driver.

2. Digital Enable (DEN)

Set the bit to 1 to enable the digital function.

3. Writing Data (DATA)

Write 1 to the data register to drive High (3.3V), or 0 to drive Low (0V).

```
void LED_Init(void) {
    volatile uint32_t delay;
    SYSCTL_RCGCGPIO_R |= 0x20; // Clock Port F delay
    delay = SYSCTL_RCGCGPIO_R;

    // PF2 (Blue LED) as Output
    GPIO_PORTF_DIR_R |=
    0x04;
    GPIO_PORTF_DEN_R |= 0x04;
}

void LED_Toggle(void) {
    // Toggle PF2 using XOR
    GPIO_PORTF_DATA_R ^=
    0x04;
}
```

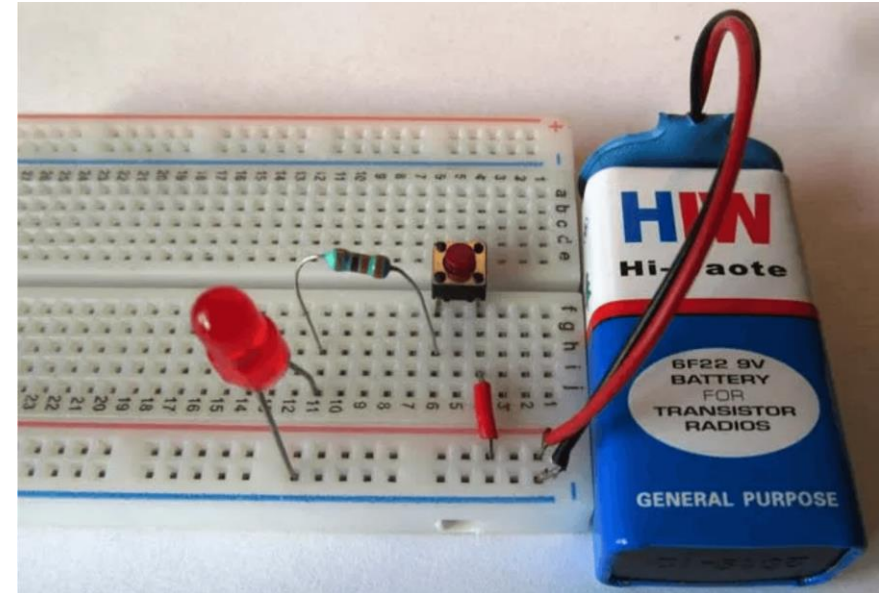
Building an LED Circuit

Bill of Materials

Red LED (Standard)	470Ω Resistor
Jumper Wires	Breadboard

Wiring Steps (Active High)

- 1 Identify the LED leads: **Long Leg** is Anode (+), **Short Leg** is Cathode (-).
- 2 Insert the LED into the breadboard.
- 3 Connect the **470Ω Resistor** between the GPIO Pin (e.g., PF1) and the Anode.
- 4 Connect the Cathode directly to **Ground (GND)**.



Polarity Check

If you trimmed the legs, look at the rim of the LED. The side with the **Flat Spot** is the Cathode (Negative). Connecting it backwards won't damage it immediately, but it won't light up.

Controlling Multiple LEDs

1. Port-Wide Control

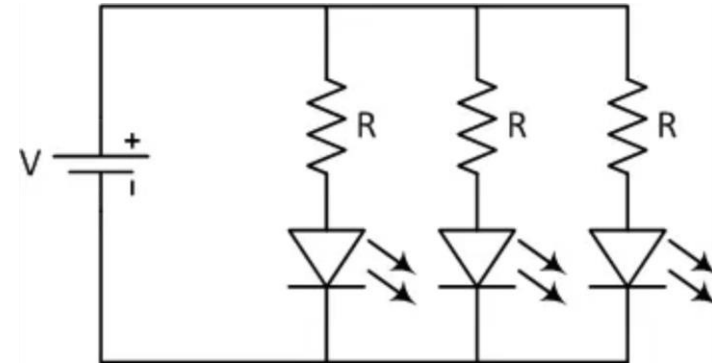
Instead of toggling pins individually, you can write to the entire **Data Register**. This allows simultaneous state changes for multiple LEDs in a single clock cycle.

2. Independent Resistors

Each LED must have its own current-limiting resistor. Sharing one resistor for parallel LEDs leads to uneven brightness due to slight variations in V_f .

⚡ Power Constraint

Total Port Current: The microcontroller has a maximum current limit for the entire port (e.g., 100mA). Ensure the sum of all LED currents does not exceed this value.



Simultaneous Toggle Example

// Turn on Red (PF1) and Blue (PF2) // Binary: 0000 0110 -> Hex: 0x06

```
GPIO_PORTF_DATA_R |= 0x06;
```

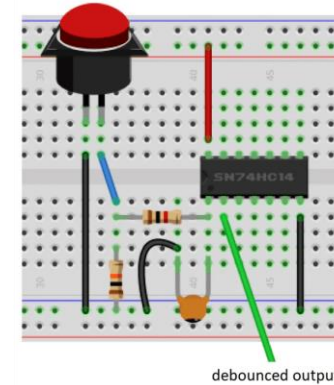
// Turn off both simultaneously

```
GPIO_PORTF_DATA_R &= ~0x06;
```

Building a Simple Switch-LED Application

- 1. Input** Read the switch state from **PF4**.
- 2. Logic** Check if the switch is pressed (Logic 0 due to Pull-Up).
- 3. Output** Turn ON the LED on **PF2** if pressed, otherwise turn OFF.

```
while(1) {  
    in = GPIO_PORTF_DATA_R & 0x10; // Read PF4  
    if(in == 0x00) {  
        GPIO_PORTF_DATA_R |= 0x04; } else { GPIO_PORTF_DATA_R &= ~0x04; }  
}
```



System Integration: This simple loop creates a "Momentary Switch" behavior. The LED mirrors the physical action of the user in real-time.

State Machine-Based Debouncing

1. Idle (Released)

Input is High. Waiting for falling edge.

2. Validate Press

Input went Low. Wait 10ms. If still Low, confirm Press.

3. Active (Pressed)

Input is Low. Action Triggered. Waiting for rising edge.

4. Validate Release

Input went High. Wait 10ms. If still High, return to Idle.

Why use a State Machine?

- 1 Non-Blocking Operation:** Unlike simple delays, an FSM can be implemented in a timer interrupt (e.g., every 10ms), allowing the main CPU to do other work.
- 2 Event Distinction:** Clearly separates "Press" events from "Release" events, enabling features like "Long Press" or "Double Click".
- 3 Scalability:** A single timer interrupt can update the state machines for multiple buttons simultaneously.

Using Interrupts for Switch Input

Polling vs. Interrupts

Polling: Like checking your mailbox every 5 minutes to see if mail has arrived. Wastes time and energy.

Interrupts: Like the mail carrier ringing your doorbell. You only stop what you're doing when there is actually mail.



Efficiency: CPU runs other code or sleeps until the event occurs.



Latency: Immediate response to the switch press (typically within microseconds).

Configuration Registers

GPIOIS	Interrupt Sense: 0 for Edge, 1 for Level.
GPIOIBE	Interrupt Both Edges: 0 for Single, 1 for Both.
GPIOIEV	Interrupt Event: 0 for Falling, 1 for Rising.
GPIOIM	Interrupt Mask: Set to 1 to Enable.

The ISR (Interrupt Service Routine)

```
void GPIOPortF_Handler(void){  
    // 1. Acknowledge (Clear) Flag  
    GPIO_PORTF_ICR_R = 0x10;  
    // 2. Execute Logic  
    ProcessSwitchPress();  
}
```


Controlling LED Brightness with PWM

Pulse Width Modulation (PWM)

Digital pins are either ON (3.3V) or OFF (0V). Switching rapidly simulates intermediate brightness.

Duty Cycle

Percentage of time the signal is High within one period.

$$\text{Duty Cycle (\%)} = (T_{\text{ON}} / T_{\text{PERIOD}}) \times 100$$

Persistence of Vision

At frequencies above ~50Hz, the eye averages pulses into continuous light.

10% Duty Cycle

Dim

50% Duty Cycle

Medium

90% Duty Cycle

Bright

Building a Traffic Light System

1. System Requirements		
Sequence	Go → Warn → Stop	
Go Time		5 Seconds
Warn Time		2 Seconds
Stop Time		5 Seconds

2. Hardware Mapping		
Red Light		PF1 (0x02)
Yellow Light		PF3 (0x08)
Green Light		PF3+PF2 (0x0C)

Table-Driven FSM Structure

```
struct State { uint32_t Out; uint32_t Time; const struct State *Next; };
typedef const struct State STyp;
```

State Name	Output (Hex)	Time	Next State
goS	0x0C (Green)	500	warnS
warnS	0x08 (Yellow)	200	stopS
stopS	0x02 (Red)	500	goS

GPIO Power and Current Limitations

⬆️ Current Sourcing

The MCU pin provides power (V_{DD}) to the load. Current flows **out** of the pin.
Logic High = Current Source.

⬇️ Current Sinking

The MCU pin provides a path to Ground (GND). Current flows **into** the pin from the load.
Logic Low = Current Sink.

The Hierarchy of Limits (Typical Values)

Single Pin	8mA	Max 8mA
Entire Port	Sum of Pins	Max 100mA
Total Package	Total Chip	Max 250mA

Protection Strategy: Never drive high-power devices (Motors, Relays, High-Power LEDs) directly from a GPIO pin. Use a transistor (BJT or MOSFET) or a driver IC to handle the load current.

Handling Noise & EMI

Sources of Noise

Electromagnetic Interference (EMI)

Radiated noise from nearby high-current devices like motors, relays, or wireless transmitters. Can induce false signals on GPIO lines.

Crosstalk

Signal coupling between adjacent parallel wires. A fast-switching signal on one wire can induce a voltage spike on its neighbor.

Ground Bounce

Transient shifts in the ground voltage level caused by rapid switching of large currents, potentially causing logic errors.

Mitigation Strategies



Hardware Filtering

Use **Decoupling Capacitors** (0.1μF) close to power pins. Add **RC Low-Pass Filters** on input lines to suppress high-frequency noise.



Software Filtering

Implement **Digital Filtering** or **Majority Voting** logic. Read the input multiple times; only accept the state if N consecutive reads match.



Physical Layout

Use **Twisted Pair** cables for long runs. Keep signal lines away from high-power lines. Use a solid Ground Plane on the PCB.

Optimizing GPIO Operations

⚠ Traditional RMW

Standard bit manipulation requires a **Read-Modify-Write** sequence. The CPU reads the port, masks the bits, and writes it back.

```
// Set Bit 0 High
DATA |= 0x01;

// Assembly:
LDR R0, [Addr] ; Read
ORR R0, #0x01 ; Modify
STR R0, [Addr] ; Write
```

Risk: Race Condition

If an interrupt occurs after the **Read** but before the **Write**, and the ISR modifies the same port, the ISR's changes will be overwritten when the main thread resumes.

✓ Bit-Specific Addressing

The hardware maps 256 address aliases to the data register. Writing to a specific alias affects **only** the bits enabled by the address offset.

```
// Write to Alias for Bit 0
DATA_BIT0 = 0x01;

// Assembly:
MOV R0, #0x01
STR R0, [Alias] ; Atomic Write
```

Benefit: Thread Safety

The operation is **Atomic** (single instruction). It is impossible for an interrupt to occur "in the middle" of the write. No need to disable interrupts (Critical Section).

Best Practices for GPIO Initialization



The Ritual

1. Clock First

Enable the port clock before accessing registers.

2. Wait for Stability

Wait 3–5 clock cycles for hardware to stabilize.

3. Unlock Only if Needed

Unlock only protected pins like PF0 or PD7.



Electrical Safety

Define Inputs

Do not leave inputs floating; enable PUR or PDR.

Check Drive Strength

Use 2mA default; raise only if the load requires it.

Critical Warning

Do not exceed port or package current limits.



Code Quality

No Magic Numbers

Use symbolic constants (e.g., LED_BLUE).

Atomic Operations

Use bit-specific ops to avoid race conditions.

Modular Init

Encapsulate init in functions like PortF_Init().

Debugging GPIO Problems



No Output

Root Cause Analysis

1. **Direction (DIR)** bit is 0 (Input).
2. **Digital Enable (DEN)** bit is 0.
3. **Clock (RCC)** not enabled for port.

Corrective Action

Verify initialization order: Clock → Delay → DIR → DEN. Check LED polarity (Anode vs Cathode).



Erratic Input

Root Cause Analysis

1. **Floating Pin:** No Pull-Up/Down resistor.
2. **Noise:** Switch bouncing or EMI.
3. **Loose Connection:** Breadboard contact.

Corrective Action

Enable internal **PUR/PDR** registers. Add external 10kΩ resistor. Implement software debouncing.



Stuck Pin

Root Cause Analysis

1. **Protected Pin:** NMI (PF0) or JTAG (PC0-3) pins are locked by default.

Corrective Action

Write magic number 0x4C4F434B to **GPIO_LOCK**, then set bit in **GPIO_CR** to unlock before modifying settings.

Testing GPIO Functionality

Testing Strategy

01 Static Verification

Write a minimal program that sets the pin High or Low permanently. Measure the voltage at the pin relative to GND using a multimeter.

02 Dynamic Toggling

Toggle the pin inside a loop with a large delay (e.g., 1 second). Verify the state change visually (LED) or with a multimeter.

03 Software Loopback

Physically connect an Output pin to an Input pin. Write a test script to drive the Output and verify the Input matches.

Essential Tools

Multimeter

Voltage Levels, Continuity

Logic Analyzer

Timing, PWM Duty Cycle

Debugger

Register Inspection

Oscilloscope

Signal Integrity, Noise

Loopback Test Snippet

```
// Connect PF1 (Out) to PF0 (In)
GPIO_PORTF_DATA_R |= 0x02; delay(10);
// Check PF0
if((GPIO_PORTF_DATA_R & 0x01)==0x01) TestPassed(); else TestFailed();
```


Summary of Port Architecture

System Architecture

■ Memory Mapped I/O

GPIO ports share the processor address space and are accessed with normal load/store instructions.

■ Bus Matrix

AHB-based bus gives low-latency, single-cycle access to port registers.

■ Clock Gating

Port clocks are disabled by default; enable via RCGCGPIO before using the port.

Register Interface

■ Configuration Registers

DIR sets direction (0=in,1=out); **DEN** enables digital logic; **AFSEL** selects alternate functions.

■ Data Register

DATA reads inputs or writes outputs; supports bit-masked addressing.

Performance Insight

Bit-Specific Addressing lets software atomically modify individual bits without disabling interrupts, avoiding race conditions.

Summary of Switch and LED Interfacing

Hardware Layer

Switch Interfacing

Defined States:	Always use Pull-Up or Pull-Down resistors to prevent floating inputs.
Mechanical Reality:	Switches bounce. Hardware (RC filters) or software debouncing is mandatory.

LED Interfacing

Current Limiting:	Always use a series resistor to protect the LED and the GPIO pin.
Power Limits:	Respect the maximum current ratings per pin (2-8mA) and per port.

Software Layer

Input Logic

Masking:	Use bitwise AND (&) to isolate specific pins from the data register.
Debouncing:	Implement delays or state machines to filter out transient noise (10-20ms).

Output Logic

Bit Manipulation:	Use Read-Modify-Write (OR/AND) to change bits without affecting others.
Atomicity:	Prefer bit-specific addressing to prevent race conditions in interrupt-heavy systems.

Real-World Design Considerations



Circuit Protection

ESD & Transients

Use TVS diodes or series resistors (e.g., 100Ω) on exposed GPIOs to protect against static discharge.

Isolation

Use Optocouplers when interfacing with high-voltage systems (e.g., motors, relays) to protect the MCU.



Power Efficiency

Unused Pins

Never leave pins floating. Configure as Output Low or Input with Pull-Up/Down to prevent leakage current.

Drive Strength

Use the lowest sufficient drive strength (2mA) to reduce EMI and power consumption.



Maintainability

Heartbeat LED

Always include a dedicated status LED to verify the CPU is running and the main loop is active.

Test Points

Add physical test points (loops or pads) for critical signals (PWM, Interrupts) for logic analyzer access.

Advanced GPIO Topics



Edge-Triggered Interrupts

Move beyond polling. Configure GPIO hardware to detect rising, falling, or both edges. The CPU is notified immediately via an Interrupt Service Routine (ISR) only when the state changes.

Benefit

Eliminates wasted CPU cycles checking for button presses, allowing the main loop to focus on other tasks.



Direct Memory Access (DMA)

For high-speed data acquisition, the DMA controller can transfer data directly between GPIO ports and RAM. The CPU configures the transfer and is then free to perform calculations.

Benefit

Enables high-throughput I/O (e.g., parallel camera interface) without stalling the processor.



Wake-Up Sources

In deep sleep modes, the CPU clock is stopped to save power. GPIO edge detection logic remains active and can generate a "Wake-Up" signal to return the system to active mode.

Benefit

Critical for battery-powered IoT devices that spend 99% of their time sleeping.

GPIO in IoT Systems

Sensing (Inputs)

- 01 Security:** Reed switches (doors/windows) and PIR motion sensors are simple binary GPIO inputs.
- 02 Safety:** Water leak detectors and smoke alarm triggers often output a simple High/Low logic signal.

Actuation (Outputs)

- 01 Smart Plugs:** A GPIO pin drives a transistor, which energizes a relay to switch 120V/240V AC power.
- 02 Lighting:** PWM-capable GPIO pins drive MOSFETs for dimmable LED strips and smart bulbs.

From Pin to Cloud

In an IoT architecture, the GPIO Event is the trigger for network activity. Latency is critical: hardware interrupts ensure the cloud is notified milliseconds after a physical event occurs.

Physical Event



GPIO Interrupt



Wake MCU



MQTT Publish

Real-World Example: Smart Home Control

GPIO Inputs

Reed Switch

Magnetic sensor on doors/windows. Reads logic **LOW** when closed, **HIGH** when open.

PIR Sensor

Passive Infrared motion detector. Outputs a digital **HIGH** pulse when movement is detected.

Wall Switch

Standard mechanical rocker switch. Requires **Debouncing** logic to prevent flickering.

MCU Logic

State Machine

Tracks system mode: **ARMED**, **DISARMED**, or **ALARM**.

Signal Processing

Software filters ignore noise. Example: motion must be present for a short duration to trigger actions.

Automation

Timers automatically turn off outputs after a set duration (e.g., Hallway light auto-off).
Lecture 5: I/O Ports, Switch and LED Interfacing

GPIO Outputs

Relay Module

Opto-isolated relay to switch mains lighting. Driven by GPIO **HIGH**.

Piezo Buzzer

Alarm siren. Toggled at 2kHz frequency (PWM or software loop).

Status LED

RGB LED indicating system health. Green = Ready, Red = Armed, Blinking = Alarm.

Advanced Debugging Techniques



Electrical Visibility

Logic Analyzer

Captures digital states over time. Essential for verifying timing protocols (I2C, SPI) and measuring pulse widths.

Oscilloscope

Visualizes analog voltage. Use this to detect noise, ringing, slow rise times, or "runt" pulses.



Execution Visibility

Hardware Breakpoints

Stops the CPU at a specific instruction address without modifying flash memory.

Data Watchpoints

Triggers a halt when a specific memory address is read or written. Good for catching stack/variable corruption.



Functional Visibility

GPIO Profiling

Set a spare GPIO high at the start of an ISR and low at the end. Measure the pulse width on a scope to determine CPU load.

Circular Buffer Log

Log events to a RAM array instead of UART printf. Minimal overhead; dump the buffer later via debugger.

Optimizing GPIO Performance

Bus Architecture

AHB vs. APB: Use the Advanced High-performance Bus (AHB) aperture for GPIO ports. It connects directly to the CPU core, enabling single-cycle access.

```
// Use AHB Base Address#define PORTF_AHB  
0x4005D000// vs Legacy APB#define PORTF_APB  
0x40025000
```

Impact: 2x Speedup

Bit-Specific I/O

Atomic Access: Use address aliasing to write to specific bits. This avoids the Read-Modify-Write sequence (LDR, ORR, STR), reducing instruction count.

```
// 1 Cycle Write PF2_BIT = 0x04; // vs 3 Cycle RMW  
DATA |= 0x04;
```

Impact: Zero Latency

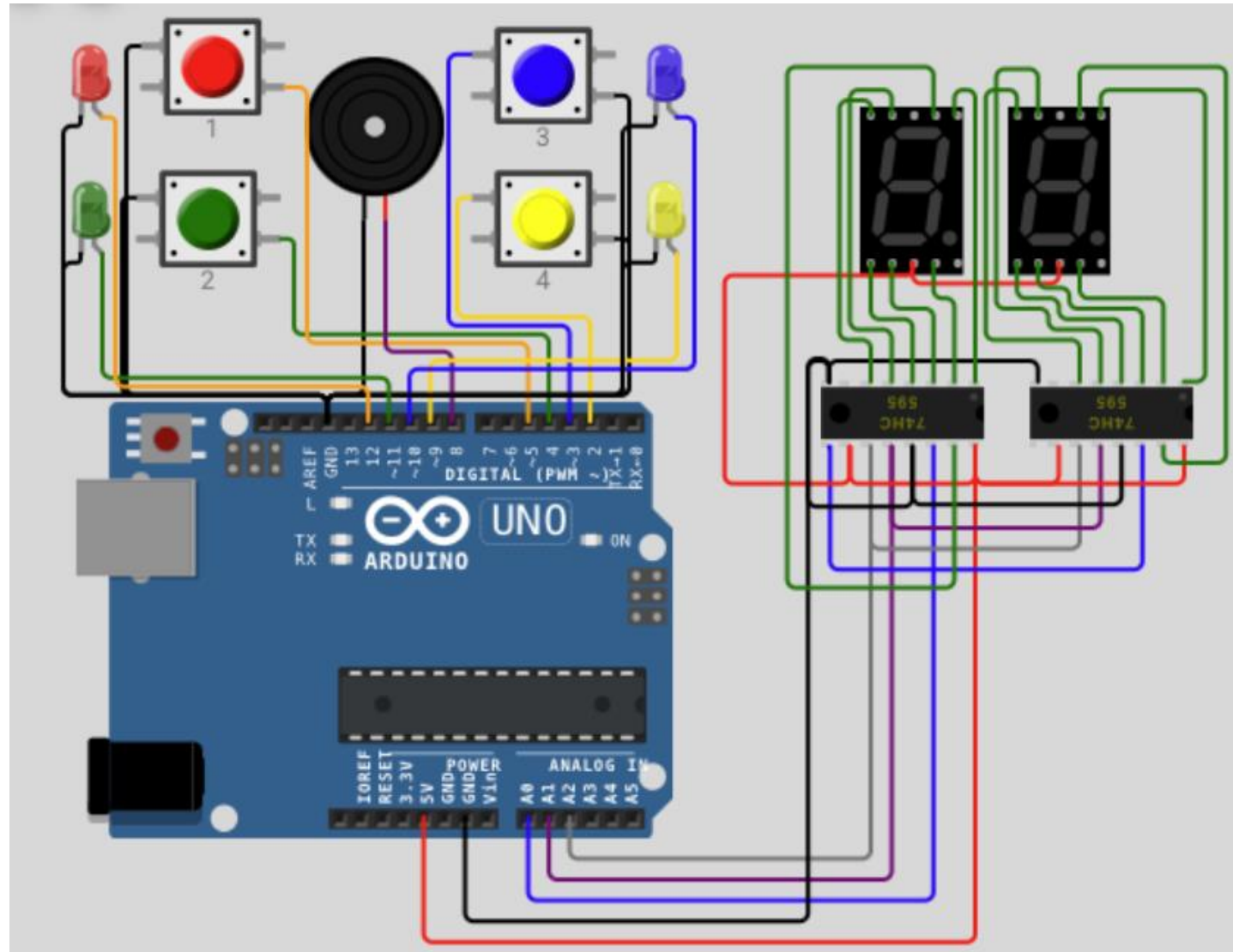
Direct Access

Bypass HAL: Hardware Abstraction Layers (HAL) add function call overhead and safety checks. For critical loops, access registers directly via pointers.

```
// Direct Pointer Access *GPIO_DATA = 0xFF; // vs  
Function Call GPIO_PinWrite(PORTF, 0xFF);
```

Impact: No Overhead

Virtual lab session recap: Simon game



Virtual lab session: Simon game

- Can you mimic a broken button (switch)—one that sometimes doesn't respond when you press it?
- And one that sometimes respond when you don't press it?

Virtual lab session: Simon game

- Implement the logic to handle the broken button or switch?

Thank you!